

Linguagens e Ambientes de Programação (Aula Teórica 15)

LEI - Licenciatura em Engenharia Informática

João Costa Seco (joao.seco@fct.unl.pt)



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Agenda

- Mutabilidade
- Hashtables
- Arrays
- Memoisation

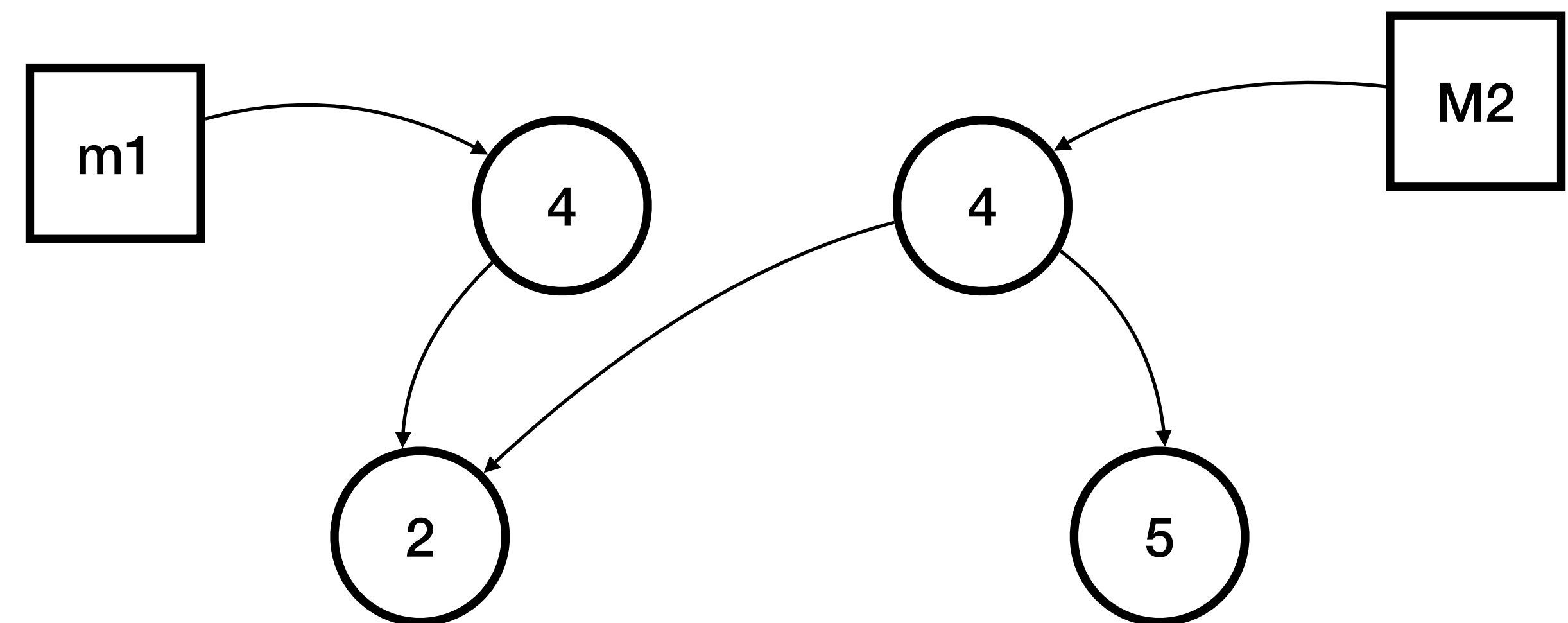
Data Structures in OCaml

- Purely functional data structures are immutable vs. mutable ephemeral data structures in imperative languages.
- When changing a data structure, the previous state is still available.
- There is no copy operation to duplicate the internal structures, but they share the internal structural elements and data that did not change.
- A Map in OCaml is an AVL (a balanced binary search tree)

```
module IntMap = Map.Make(Int)

let m1 =
  IntMap.empty
|> IntMap.add 2 "Node 2"
|> IntMap.add 4 "Node 4"

let m2 = IntMap.add 5 "Node 5" m1
```



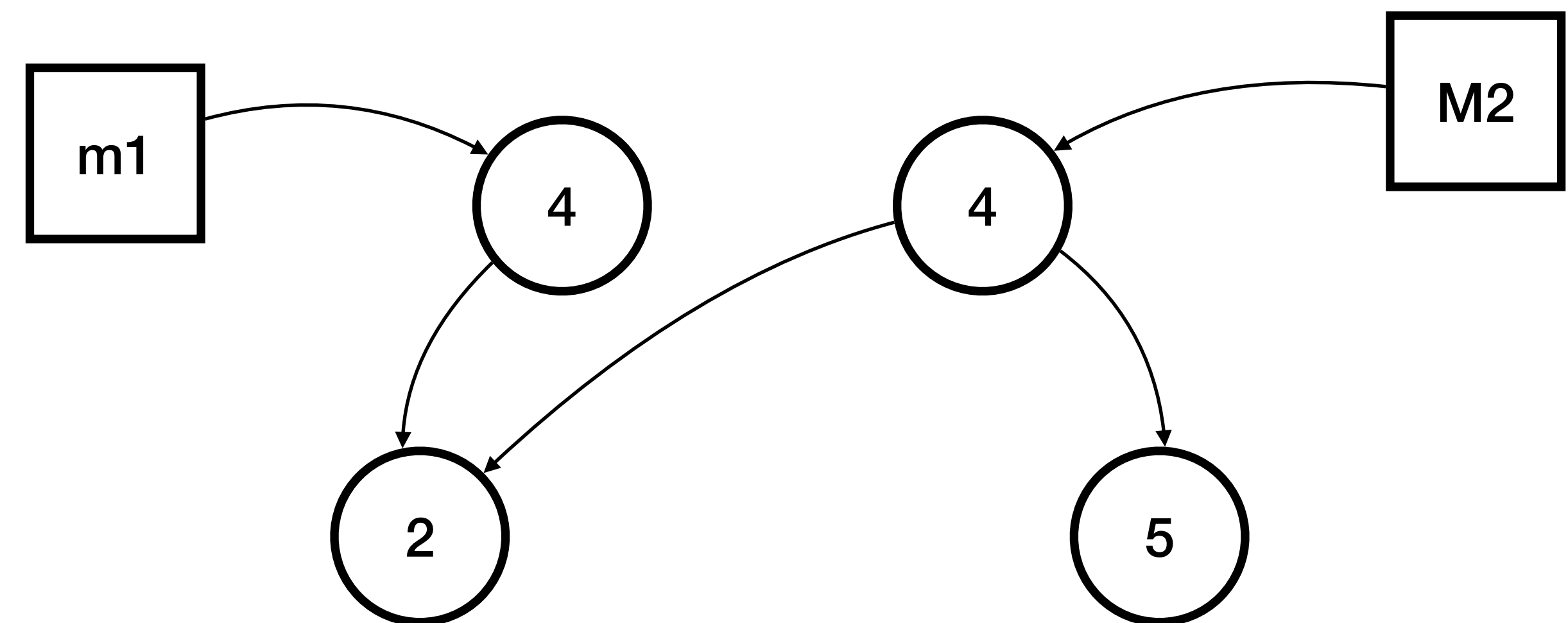
Data Structures in OCaml (Persistent data structures)

- Data structures that provide a new entry point on change and preserve the older version while reusing most of the structural and data parts are called persistent data structures. (Git is a persistent data structure)
- Advantages: undo/rollback/history/snapshot isolation, no locking when reading/changing MVCC (Multi-Version Concurrency Control) in PostgreSQL, CouchDB, etc.
- Disadvantages: some memory overhead when compared to simpler ephemeral versions. Garbage collection is fast for short-lived objects (the compiler course will tackle this).

```
module IntMap = Map.Make(Int)

let m1 =
  IntMap.empty
  |> IntMap.add 2 "Node 2"
  |> IntMap.add 4 "Node 4"

let m2 = IntMap.add 5 "Node 5" m1
```



Mutable data structures in OCaml

- OCaml provides mutable data structures. It is a good practice to use them as infrastructure and avoid interference with the functional algorithms.
 - References
 - Array
 - Mutable Records
 - Hashtbl
 - Queue
 - Stack

Array

- The basic mutable data structure. Implemented in a contiguous block of memory with random access. All elements are of the same size.

```
let numbers = [| 1; 2; 3 |]  
  
let _ = assert (numbers.(0) = 1)  
  
numbers.(1) <- 10  
  
let _ = assert (numbers.(1) = 10)
```


Mutable records

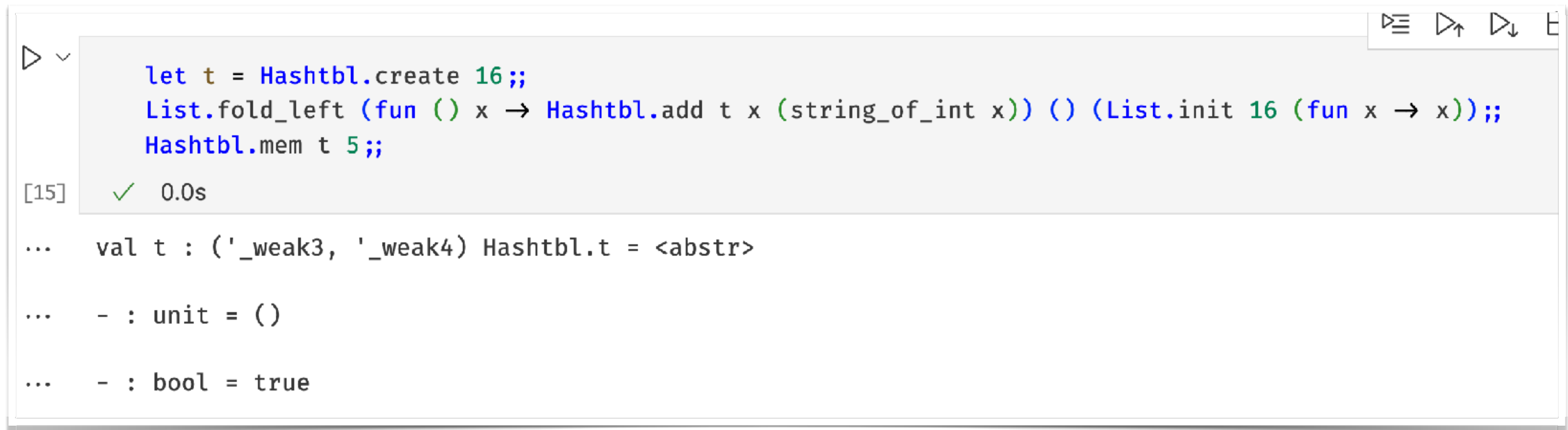
- Record fields can be declared mutable and avoid the clutter of using references on top of immutable records.

```
type user = {  
  id : int; (* Immutable *)  
  mutable nickname : string; (* Mutable *)  
}  
  
let u = { id = 1; nickname = "Camel" }  
u.nickname <- "Alpaca"; (* This works *)  
u.id <- 2; (* This does not work *)
```

- Examples, cache of results, circular data structures, sharding, etc.

Hashtbl

- O módulo Hashtbl tem uma interface genérica com uma função de hash built-in, e também tem um functor Make que permite a sua customização.



```
let t = Hashtbl.create 16;;
List.fold_left (fun () x → Hashtbl.add t x (string_of_int x)) () (List.init 16 (fun x → x));;
Hashtbl.mem t 5;;

[15] ✓ 0.0s

... val t : ('_weak3, '_weak4) Hashtbl.t = <abstr>

... - : unit = ()

... - : bool = true
```


Hashtbl

- O módulo Hashtbl tem uma interface genérica com uma função de hash built-in, e também tem um functor Make que permite a sua customização.

```
module PairHash = struct
  type t = string * int
  let equal (a1, b1) (a2, b2) = a1 = a2 && b1 = b2
  let hash (a, b) = Hashtbl.hash (a, b)
end
```

```
module PairHashtbl = Hashtbl.Make(PairHash)
```

[4] ✓ 0.0s

```
... - : bool = true
```

```
... module PairHash :
  sig
    type t = string * int
    val equal : 'a * 'b → 'a * 'b → bool
    val hash : 'a * 'b → int
  end

... module PairHashtbl :
  sig
    type key = PairHash.t
    type 'a t = 'a Hashtbl.Make(PairHash).t
    val create : int → 'a t
    val clear : 'a t → unit
    val reset : 'a t → unit
    val copy : 'a t → 'a t
    val add : 'a t → key → 'a → unit
    val remove : 'a t → key → unit
    val find : 'a t → key → 'a
    val find_opt : 'a t → key → 'a option
    val find_all : 'a t → key → 'a list
    val replace : 'a t → key → 'a → unit
    val mem : 'a t → key → bool
    val iter : (key → 'a → unit) → 'a t → unit
    val filter_map_inplace : (key → 'a → 'a option) → 'a t → unit
    val fold : (key → 'a → 'b → 'b) → 'a t → 'b → 'b
    val length : 'a t → int
    val stats : 'a t → Hashtbl.statistics
    val to_seq : 'a t → (key * 'a) Seq.t
    val to_seq_keys : 'a t → key Seq.t
    val to_seq_values : 'a t → 'a Seq.t
    val add_seq : 'a t → (key * 'a) Seq.t → unit
    val replace_seq : 'a t → (key * 'a) Seq.t → unit
    val of_seq : (key * 'a) Seq.t → 'a t
  end
```

Memoization

Memoization

```
let rec fib n = if n < 2 then 1 else fib (n - 1) + fib (n - 2)
```

```
let _ = fib 45
```

[7] ✓ 38.2s

```
... val fib : int → int = <fun>
```

```
... - : int = 1836311903
```


Recursão pela memória (Landin's knot)

- A implementação de recursão pode ser feita sem recurso à recursão nativa.
- Uma variável de estado (ref) pode guardar a continuação
- Bom para substituir e interceptar chamadas (ex: para implementar memoization)
- Bom para implementar trace em debuggers



```
let rec fact_rec n = if n = 0 then 1 else n * fact_rec (n - 1)

let fact0 = ref (fun x → x + 0)

let fact n = if n = 0 then 1 else n * !fact0 (n - 1);;
fact0 := fact

let _ = fact 5
```

[19]

✓ 0.0s

```
... val fact_rec : int → int = <fun>
... val fact0 : (int → int) ref = {contents = <fun>}
... val fact : int → int = <fun>
... - : unit = ()
... - : int = 120
```

Recursão pela memória (Landin's knot)

- A implementação pode ser recursiva
- Uma implementação pode ser iterativa
- Bom exemplo de interação entre memória e recursão (ex: pila de memória)
- Bom exemplo de uso de ponteiros

```
#include <stdio.h>

int (*fib_ptr)(int);

int fib_body(int n) {
    if (n <= 1) return n;
    return fib_ptr(n - 1) + fib_ptr(n - 2);
}

int main() {
    fib_ptr = fib_body;

    printf("%d\n", fib_ptr(10));
    return 0;
}
```

C

c (n - 1)

;

Memoization

```
let fib_memo n =
  let memo = Hashtbl.create 16 in
  let rec fib_memo' n =
    if n < 2 then 1
    else
      match Hashtbl.find_opt memo n with
      | Some v → v
      | None →
        let v = fib_memo' (n - 1) + fib_memo' (n - 2) in
        Hashtbl.add memo n v;
        v
  in
  fib_memo' n

let _ = fib_memo 45
```

[7] ✓

... val

... - :

[8] ✓ 0.0s

... val fib_memo : int → int = <fun>

... - : int = 1836311903

- 2)

General Memoization



```
let fib_0 = ref (fun x → x)
let rec fib_rec n = if n < 2 then 1 else (!fib_0) (n - 1) + (!fib_0) (n - 2)
let _ = fib_0 := fib_rec

let _ = fib_rec 45
```

[25] ✓ 40.2s

```
... val fib_0 : ('_weak24 → '_weak24) ref = {contents = <fun>}
... val fib_rec : int → int = <fun>
... - : unit = ()
... - : int = 1836311903
```


General Memoization



```
let fib_0 = ref (fun x → x)
let rec fib_rec n = if n < 2 then 1 else (!fib_0) (n - 1) + (!fib_0) (n - 2)
let _ = fib_0 := fib_rec
```

[25]

...

...

...

...

```
let fib_hash = Hashtbl.create 16
let fib_mem =
  fun n →
    match Hashtbl.find_opt fib_hash n with
    | Some v → v
    | None → let v = fib_rec n in
              Hashtbl.add fib_hash n v;
              v
let _ = fib_0 := fib_mem
let _ = fib_rec 45
```


General Memoization

▶ ▼

[25] ✓

...

val

...

val

...

- :

...

- :

```
let fib_hash = Hashtbl.create 16
let fib_mem =
  fun n →
    match Hashtbl.find_opt fib_hash n with
    | Some v → v
    | None → let v = fib_rec n in
              Hashtbl.add fib_hash n v;
              v
let _ = fib_0 := fib_mem
let _ = fib_rec 45
```

(n - 2)

[24] ✓ 0.0s

... val fib_hash : ('_weak22, '_weak23) Hashtbl.t = <abstr>

... - :

... val fib_mem : int → int = <fun>

... - :

... - : unit = ()

... - : int = 1836311903